

P2985R0

A type trait for detecting virtual base classes

Published Proposal, 2023-09-28

Author:

[Giuseppe D'Angelo](#)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

We propose a new type trait that detects if a class is a virtual base of another class. This trait can be used in smart pointer classes, in order to constrain or optimize certain operations.

Table of Contents

1 Changelog

2 Motivation and Scope

2.1 Prior art

3 Design Decisions

3.1 Do we need this trait in the Standard Library? Can it be implemented entirely in user code?

3.2 What about classes that inherit virtually and non-virtually from the same base?

3.3 Is a class a virtual base of itself?

4 Impact on the Standard

5 Technical Specifications

6 Proposed wording

7 Acknowledgements

References

Informative References

§ 1. Changelog

- R0
 - First submission

§ 2. Motivation and Scope

We propose the addition of the `std::is_virtual_base_of` type trait to the Standard Library, that complements the existing `std::is_base_of` trait. The trait aims to detect whether a given class is a virtual base class of another one, ignoring accessibility, ambiguities and cv-qualifiers, just like `std::is_base_of`.



Detecting whether a class is a virtual or non-virtual base of another class has applications in the context of pointer conversions. Given a pointer `Derived *d` convertible to `Base *` (that is, `Base` is an accessible, non-ambiguous base of `Derived`), actually performing the conversion requires completely different implementations depending on whether `Base` is a non-virtual or a virtual base of `Derived`.

On all common ABIs, if `Base` is a non-virtual base of `Derived`, then the conversion consists in a test for `nullptr`. Otherwise, a constant offset is applied to the pointer; the offset is entirely known at compile time. The test for `nullptr` is necessary to yield `nullptr` back in that case; on common ABIs, null pointers to objects are represented by a `0` value, and we have to return `0` (= null pointer) and not `0 + offset` to honor `[conv.ptr]/3` (null pointers convert to null pointers).

If instead `Base` is a virtual base of `Derived`, then we perform the test for `nullptr` as usual. In case we don't have a null pointer, we then have to determine the address of the `Base` subobject inside the `Derived` object. The actual mechanics depend on the ABI, but this always requires inspecting `d`'s virtual table, which means examining `*d` (the *pointee*). If the pointee has already been destroyed and the pointer is dangling (an "invalid pointer value", as per [basic.stc.general]/4's definition), then this will cause problems.

Technically speaking, the result of the cast is implementation-defined; [basic.stc.general]/4 says that "Any other use of an invalid pointer value has implementation-defined behavior". (In particular, we don't think that the "Indirection through an invalid pointer value and passing an invalid pointer value to a deallocation function have undefined behavior." sentence applies here, as the indirection is necessary due to implementation/ABI requirements, and not language requirements.)

Although no implementation seems to document their behavior, on all common ABIs, the conversion of an invalid pointer value towards a virtual base causes an illegal memory access (which may segfault, or just read "garbage", etc.); while a conversion towards a non-virtual base class does not cause any issues (see the note below).



This behavior with pointer conversions is taken into account, for instance, in `std::weak_ptr`'s converting constructor. On all major implementations (libstdc++, libc++, MS-STL), `weak_ptr` is implemented with this class layout:

```
template <typename T>
class weak_ptr
{
    control_block *m_cb;
    T *m_data;
};
```

Let's now consider the move-converting constructor from a `weak_ptr<Y>` to `weak_ptr<T>`. Simplifying, the constructor is constrained on `Y*` being convertible to `T*` (cf. [util.smartptr.weak.const]/6 and [util.smartptr.shared.general]/6).

The "obvious" implementation:

```
template <typename Y>
requires std::is_convertible_v<Y *, T *>
weak_ptr(weak_ptr<Y> &&other)
: m_cb(std::exchange(other.m_cb, nullptr)),
  m_data(std::exchange(other.m_data, nullptr)) // <-- danger
{}

```

is wrong and will produce UB / crashes when converting `other.m_data` to `m_data`, in case 1) `T` is a virtual base of `Y` and 2) the managed object has already been destroyed. This is a concrete possibility -- the *raison d'être* for `weak_ptr` is that it does **not** keep the managed object alive.

A correct implementation therefore needs to check that the managed object is still alive:

```
template <typename Y>
requires std::is_convertible_v<Y *, T *>
weak_ptr(weak_ptr<Y> &&other)
: m_cb(other.m_cb),
  m_data(other.lock().get())
{
    other.m_cb = nullptr; other.m_data = nullptr;
}

```

The above snippet works, but now this is a pessimization for the "common" case when `T` is a non-virtual base of `Y`. In that case we could avoid the (relatively more expensive) `lock()` operation and just safely offset the pointer.

If an implementation wishes to optimize for this case, then they would need to detect if `T` is a virtual base class of `Y`, which is where they would use the type trait that we're proposing:

```
// simple implementation for non-virtual base
template <typename Y>
requires (std::is_convertible_v<Y *, T *> && !std::is_virtual_base_of_v<Y, T>)
weak_ptr(weak_ptr<Y> &&other)
: m_cb(std::exchange(other.m_cb, nullptr)),
  m_data(std::exchange(other.m_data, nullptr))
{}

// correct, more expensive implementation for virtual bases
template <typename Y>

```

```

requires (std::is_convertible_v<Y *, T *> && std::is_virtual_base_of_v<T,
    weak_ptr(weak_ptr<Y> &&other)
    : m_cb(other.m_cb),
      m_data(other.lock().get())
    {
        other.m_cb = nullptr; other.m_data = nullptr;
    }
}

```

NOTE: On all common Standard Library implementations there is no concern for accessing `m_data` even in case it may contain an invalid pointer value ([basic.stc.general]/4). For instance, it's simply copied as a pointer (and not `memcpy`'d or similar) in their implementation of `weak_ptr`'s move constructor. This confirms our understanding that no major compiler/ABI has any "special" behavior with regards to invalid pointer values, although they don't document their implementation-specific behavior in the area.



A similar consideration may be applied to `observer_ptr`'s converting constructor, which could be marked as `explicit` if a conversion towards a virtual base class is requested (since such a conversion may be "unsafe"). We however do not have sufficient field experience with such a change to reach any conclusions.

§ 2.1. Prior art

[Boost.TypeTraits] has shipped a `boost::is_virtual_base_of` type trait since at least Boost version 1.40 (2009). A similar implementation has been added to Qt as private API. The trait that we are proposing will however have slightly different semantics.

A trait with the same name and semantics has also been proposed by [N3987], although in a different context (reflection), and together with many more traits. We are unsure of the status of that paper, which looks more like a thought experiment than an actual proposal.

§ 3. Design Decisions

§ 3.1. Do we need this trait in the Standard Library? Can it be implemented entirely in user code?

This trait has been indeed implemented in user code, but the detection achieved this way is *incomplete* (see below). The implementation is definitely "experts-only", and therefore would benefit from being provided by the Standard Library.

The core of the detection in Boost and Qt combines these two checks:

1. whether `(Base *)std::declval<Derived*>()` is well formed. If it is, then `Base` is an unambiguous base of `Derived`. This will be true if `Base` is a virtual base class of `Derived` (and `Derived` doesn't also inherit from `Base` non-virtually). The cast notation is used in order to ignore accessibility ([expr.cast]/4).
2. whether `(Derived *)std::declval<Base*>()` is **ill** formed. This is the case if `Base` is an ambiguous or virtual base of `Derived` (again ignoring accessibility).

With the first check we establish that `Base` is not an ambiguous base, and combined with the second check, we therefore establish that it must be a virtual base.

§ 3.2. What about classes that inherit virtually and non-virtually from the same base?

The detection in usercode has a shortcoming: if `Derived` inherits virtually *and* non-virtually from `Base`, then 1. fails, but `Base` still is "a" virtual base class. Such a situation is perfectly legitimate and allowed by the core language.

Consider for instance the example in Note 6 in [class.mi]:

```
class B { /* ... */ };
class X : virtual public B { /* ... */ };
class Y : virtual public B { /* ... */ };
class Z : public B { /* ... */ };
class AA : public X, public Y, public Z { /* ... */ };
```

In this example `AA` inherits from `B` twice: once virtually (through `X` and `Y`) and once non-virtually (through `Z`). Therefore, what should `std::is_virtual_base_of_v<B, A>` yield? There are two possible lines of reasoning:

1. `B` is a virtual base class (considering all possible inheritance paths); therefore the trait should yield `true`;
2. `B` is not a virtual base class in at least one inheritance path; therefore the trait should yield `false`.

(In other words: with 1. we would be asking if `B` is a virtual base class, and with 2. we would be asking if `B` is a virtual base class and also not a non-virtual base class.)

We believe that the following considerations apply here:

- Each choice has some merits. 1. is what the core language says; however 2. is what `Boost.TypeTraits` and similar code actually detect (and have done so for a very long time). We are not aware of any implementation in user code that is able to do detection no. 1, nor viable implementation strategies.
- Whether the trait does 1. or 2. does not ultimately affect the main use case of the trait (pointer conversions). In the example, `AA*` is simply not convertible to `B*` because the conversion is ambiguous, so for those use cases it won't matter at all what the trait detects.
- `std::is_base_of` already somehow deviates from the base class definition in core language, as the core language doesn't allow a class to be a base of itself ([`class.derived.general`]/2, "A *class-or-decltype* shall denote a (possibly cv-qualified) class type that is not an incompletely defined class").

In this proposal we feel more comfortable at embracing the core language definition of virtual base class, and therefore we would propose choice no. 1 (that is, for the trait to detect that `B` is a virtual base class of `AA`), despite the prior art in Boost and Qt.

We also don't believe it is going to be necessary to offer a trait that does detection no. 2, for two reasons. First, having fine-grained detections sounds more like task for a reflection system rather than for type traits in the standard library. (For instance, there aren't multiple "versions" of `is_base_of` that check for accessibility, multiple inheritance from the same class, and so on. The trait does one thing and one thing only.) Second, as discussed above, we do not have any use case for it, and would not risk introducing two traits that will just cause confusion to end users.

§ 3.3. Is a class a virtual base of itself?

We are aware that, although no class is a base class of itself, `std::is_base_of_v<X, X>` is actually `true`. While one can consider `is_base_of` to model a general inheritance/is-A relationship between types, the check for virtual inheritance is actually much more specific, and we strongly feel that the trait should yield what the core language describes here: no class is a virtual base class of itself.

§ 4. Impact on the Standard

This proposal is a pure library extension; it adds a type trait to `<type_traits>`. There are no changes required in the core language.

Vendors are expected to implement the trait through internal compiler hooks, as they already do for other similar type traits (e.g. `std::is_base_of`), for performance, correctness and maintenance reasons.

§ 5. Technical Specifications

All the proposed changes are relative to [\[N4958\]](#).

§ 6. Proposed wording

Add to the list in [version.syn]:

```
#define __cpp_lib_is_virtual_base_of YYYYMMML // also in <type_traits>
```

Modify [meta.type.synop] as shown.

Add to first [meta.rel] block:

```
template<class Base, class Derived> struct is_base_of;  
template<class Base, class Derived> struct is_virtual_base_of;
```

And to the second block:

```
template<class Base, class Derived>  
constexpr bool is_base_of_v  
    = is_base_of<Base, Derived>::value;  
template<class Base, class Derived>  
constexpr bool is_virtual_base_of_v  
    = is_virtual_base_of<Base, Derived>::value;
```

Insert a new row in Table 49 in [meta.rel], after the one for `is_base_of`:

<code>template<class Base,</code>	<u>Base is a virtual</u>	<u>If Base and Derived are non-union</u>
<code>class Derived> struct</code>	<u>base class of</u>	<u>class types, Derived shall be a</u>
<code>is_virtual_base_of;</code>	<u>Derived</u>	<u>complete type. [Note 2: Virtual base</u>
	<u>([class.mi])</u>	<u>classes that are private, protected or</u>
	<u>without regard to</u>	<u>ambiguous are, nonetheless, virtual</u>
	<u>cv-qualifiers.</u>	<u>base classes. — end note] [Note 3: A</u>
		<u>class is never a virtual base class of</u>
		<u>itself. — end note]</u>

§ 7. Acknowledgements

Thanks to KDAB for supporting this work.

All remaining errors are ours and ours only.

§ References

§ Informative References

[Boost.TypeTraits]

Boost.TypeTraits version 1.83: is_virtual_base_of. URL:

https://www.boost.org/doc/libs/1_83_0/libs/type_traits/doc/html/boost_typetraits/reference/is_virtual_base_of.html

[N3987]

C. Silva, D. Auresco. *Yet another set of C++ type traits*. 7 May 2014. URL:

<https://wg21.link/n3987>

[N4958]

Thomas Köppe. *Working Draft, Programming Languages — C++*. 14 August 2023. URL:

<https://wg21.link/n4958>